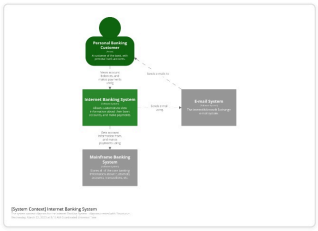
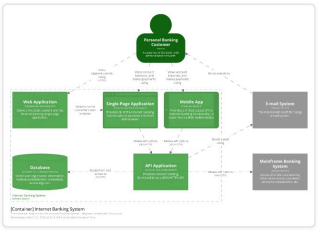
What Is a C4 Diagram?

In the world of software architecture, there are many tools and models that developers use to visualize and understand complex systems. One such tool that has gained popularity in recent years is the C4 model. At the heart of this model is the C4 diagram. A C4 diagram is a graphical representation of a software system's static structure and behavior using a collection of diagrams.

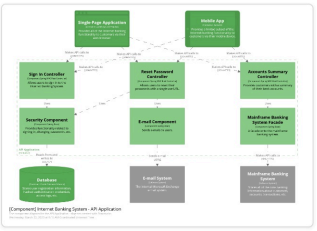
The 'C4' in C4 diagram stands for Context, Container, Component, and Code, which are the four levels of abstraction that the diagram illustrates. The C4 diagram aims to offer a simplified view of the complex structure of a software system, making it easier for various stakeholders to understand. It provides a common language for developers, stakeholders, and architects to discuss and understand the system.



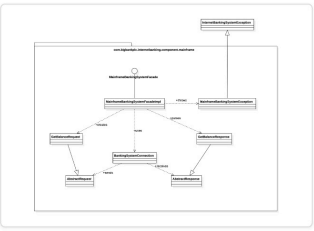
Level 1: A **System Context** diagram provides a starting point, showing how the software system in scope fits into the world around it.



Level 2: A **Container** diagram zooms into the software system in scope, showing the high-level technical building blocks.



Level 3: A **Component** diagram zooms into an individual container, showing the components inside it.



Level 4: A **code** (e.g. UML class) diagram can be used to zoom into an individual component, showing how that component is implemented.

> Source for this and the following images: C4 Model

Unlike traditional diagramming techniques, the C4 diagram focuses on the high-level system design and architecture rather than the detailed design. This approach allows the team to quickly create a shared understanding of the system, making it an ideal tool for agile teams and rapid prototyping.

This is part of a series of articles about code visualization.

In this article:

Relationship Between C4 Model and C4 Diagram

The C4 Model is a framework for visualizing the architecture of a software system. It provides a static view of a system, focusing on the structure and interaction of its various components.

On the other hand, the C4 diagram is the visual representation that encapsulates this model. In other words, the C4 model provides the theoretical basis, and the C4 diagram is the practical application. The diagram helps visualize the model, making it easier to understand and communicate the system's architecture.

The C4 model and C4 diagram are appealing to development teams because they can be used to represent systems of any size, from small software applications to large enterprise systems. This flexibility makes them an invaluable tool for software architects and developers.

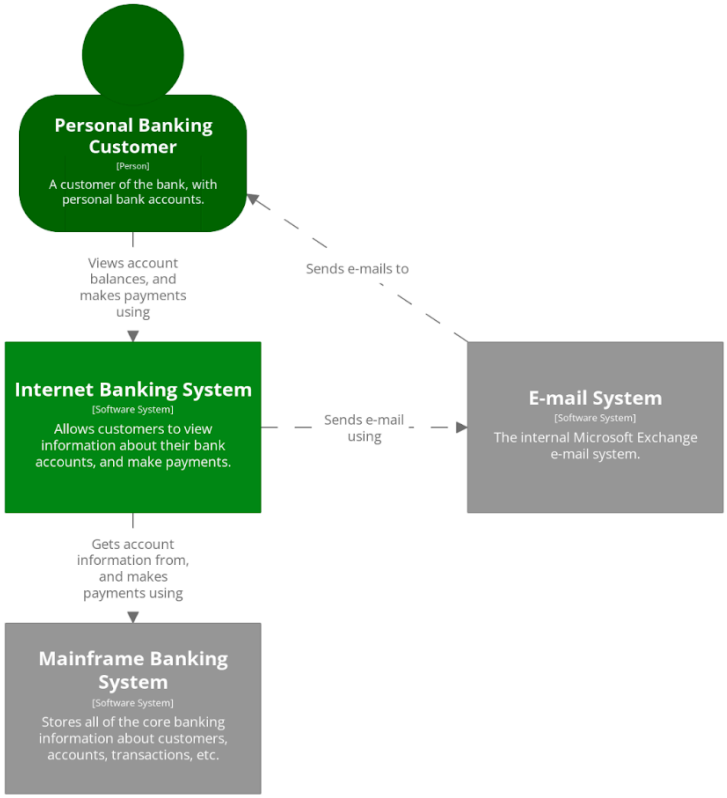
The 4 Types of C4 Diagrams

A C4 Diagram is composed of four main elements, each representing a different level of abstraction. These elements are the Context Diagram, Container Diagram, Component Diagram, and Code Diagram.

Context Diagram

The Context Diagram is the top-level view of a system. It illustrates how the system fits into the larger ecosystem. It shows the system as a single box, surrounded by its users (actors)

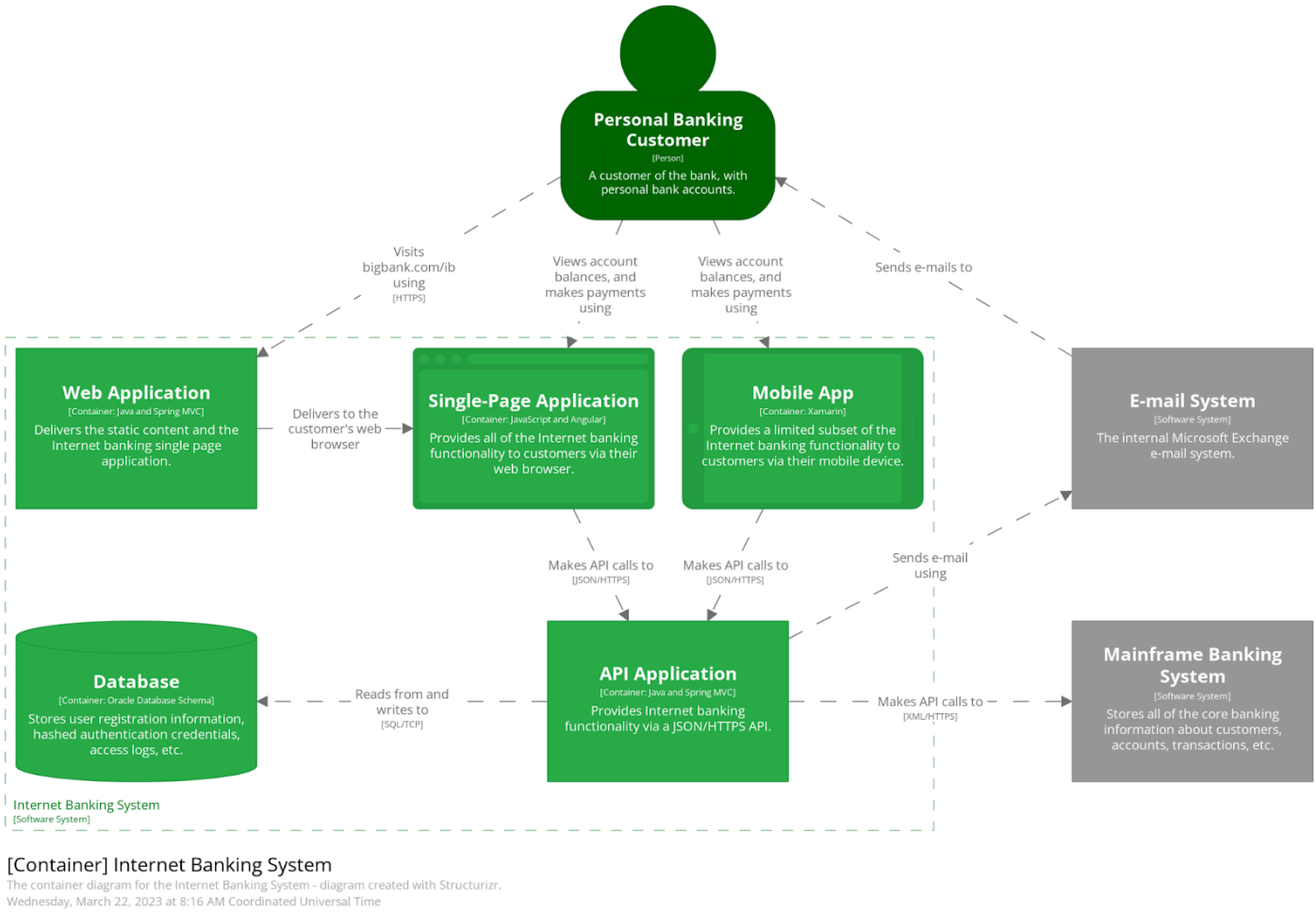
and the systems it interacts with. This diagram provides a high-level overview of the system and sets the stage for the more detailed diagrams that follow.



[System Context] Internet Banking System
The system context diagram for the Internet Banking System - diagram created with Structurizr.
Wednesday, March 22, 2023 at 8:16 AM Coordinated Universal Time

Container Diagram

The next level of abstraction is the Container Diagram. This diagram zooms into the system and shows its major software or hardware components (containers), such as web servers, databases, desktop applications, mobile apps, etc. It also shows how these containers interact with each other and with external actors. The Container Diagram provides a more detailed view of the system's architecture, but still remains high-level enough to be easily understood by non-technical stakeholders.

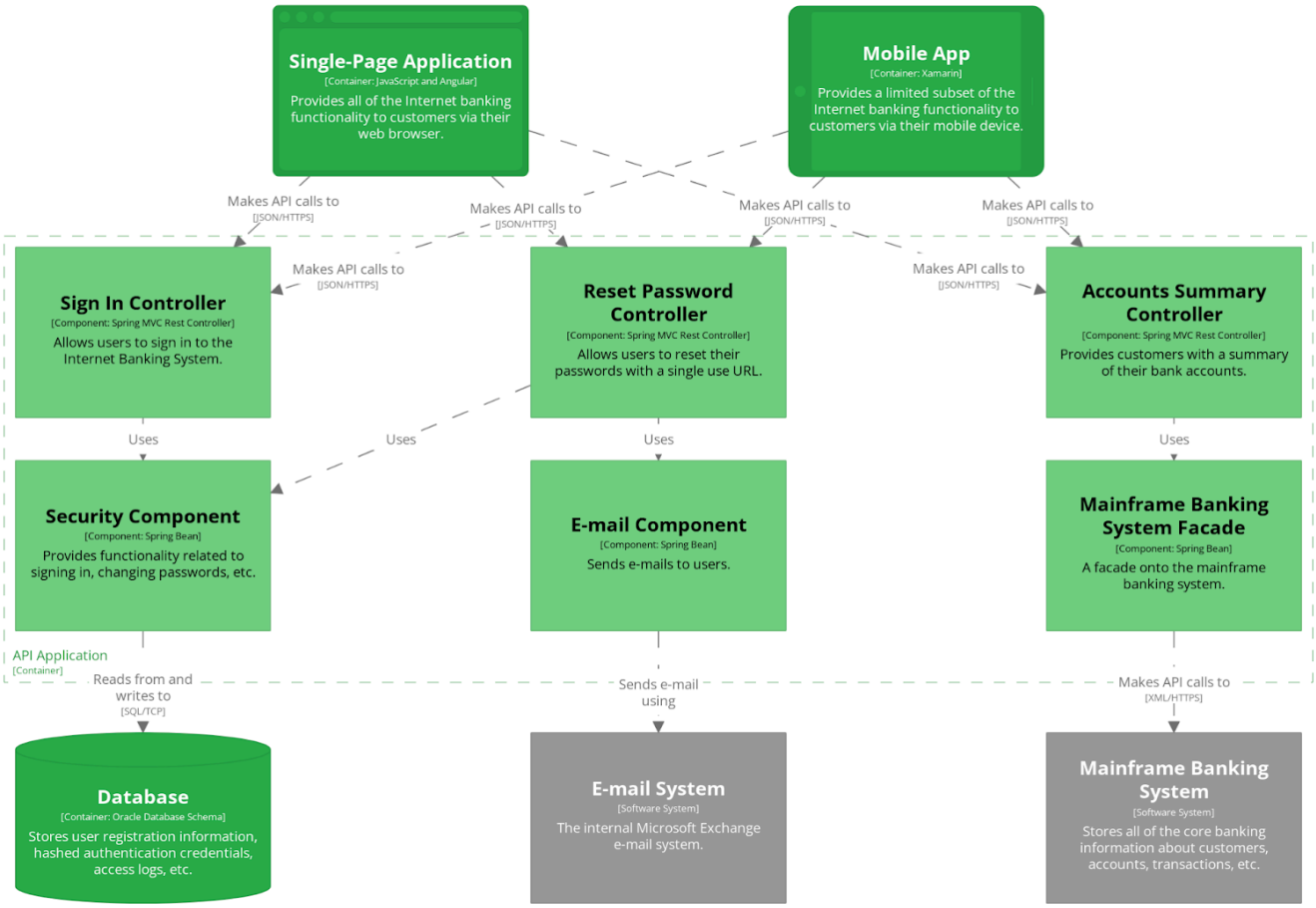


[Container] Internet Banking System

The container diagram for the Internet Banking System - diagram created with Structurizr.
Wednesday, March 22, 2023 at 8:16 AM Coordinated Universal Time

Component Diagram

The Component Diagram is the third level of abstraction in the C4 Diagram. It breaks down each container into its constituent components, showing their interactions and relationships. Components could be classes, interfaces, or larger chunks of code that provide a specific functionality. This diagram is highly useful for developers as it gives a detailed view of the system's internal structure.



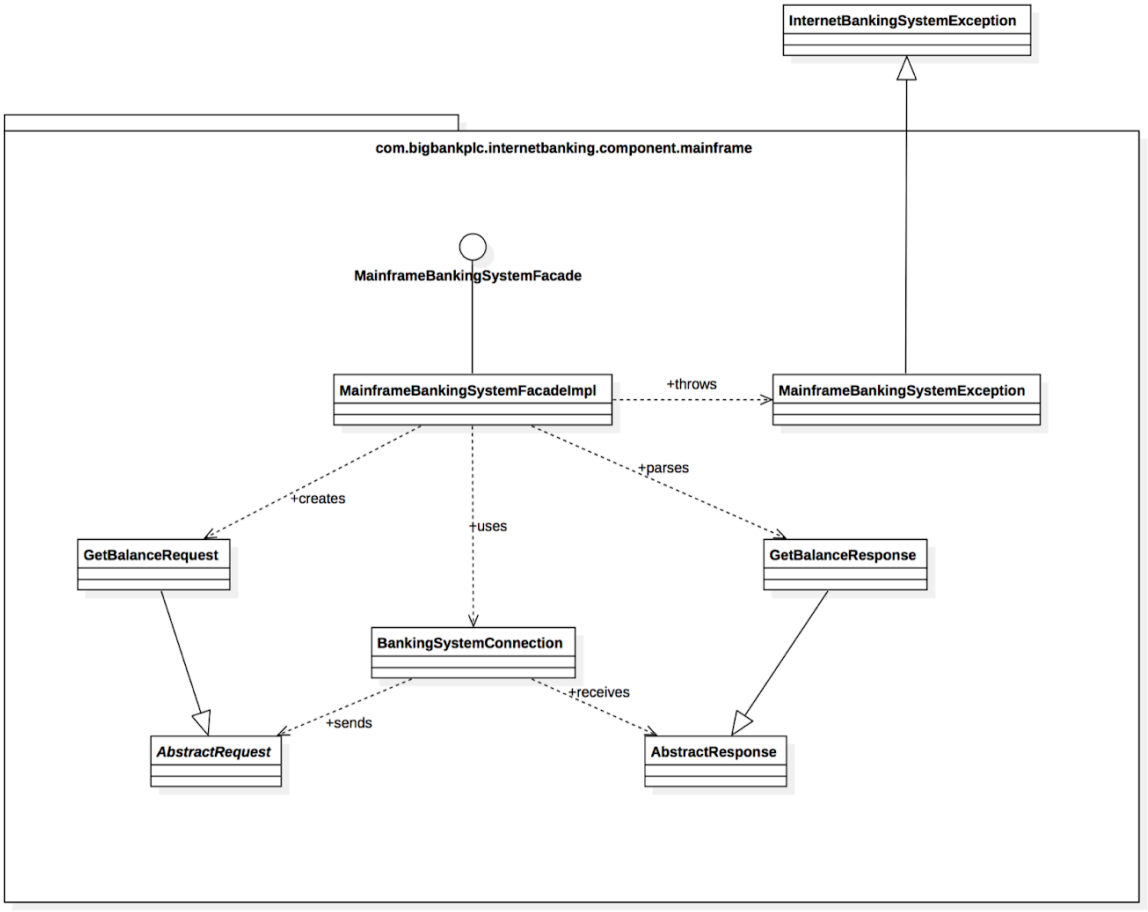
[Component] Internet Banking System - API Application

The component diagram for the API Application - diagram created with Structurizr.

Wednesday, March 22, 2023 at 8:16 AM Coordinated Universal Time

Code Diagram

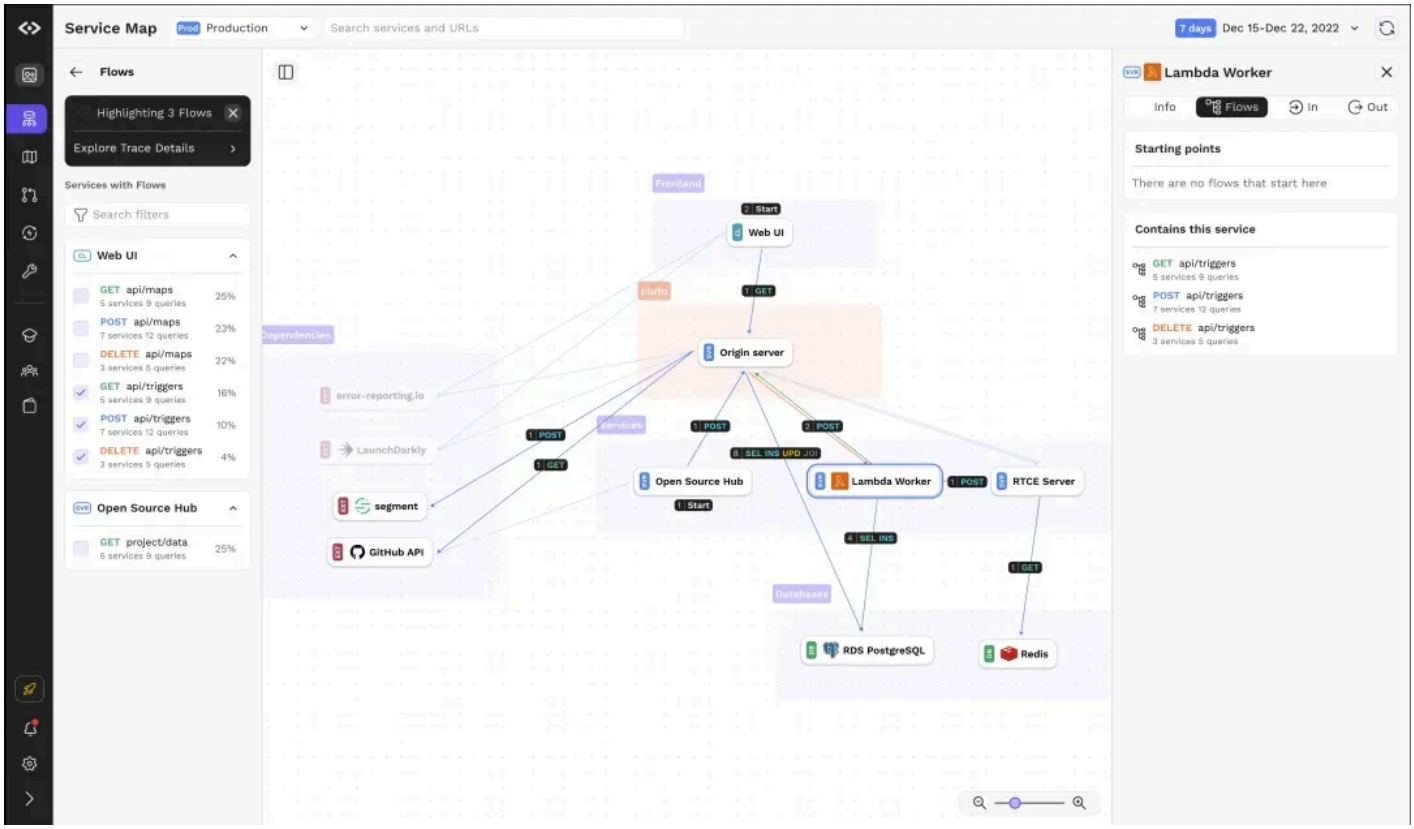
The final level of abstraction is the Code Diagram. This diagram dives into the details of individual components, showing the important classes, interfaces, and their relationships. It represents the lowest level of abstraction and is typically used by developers for designing and implementing the system.



Creating a C4 Diagram with CodeSee Code Visualization

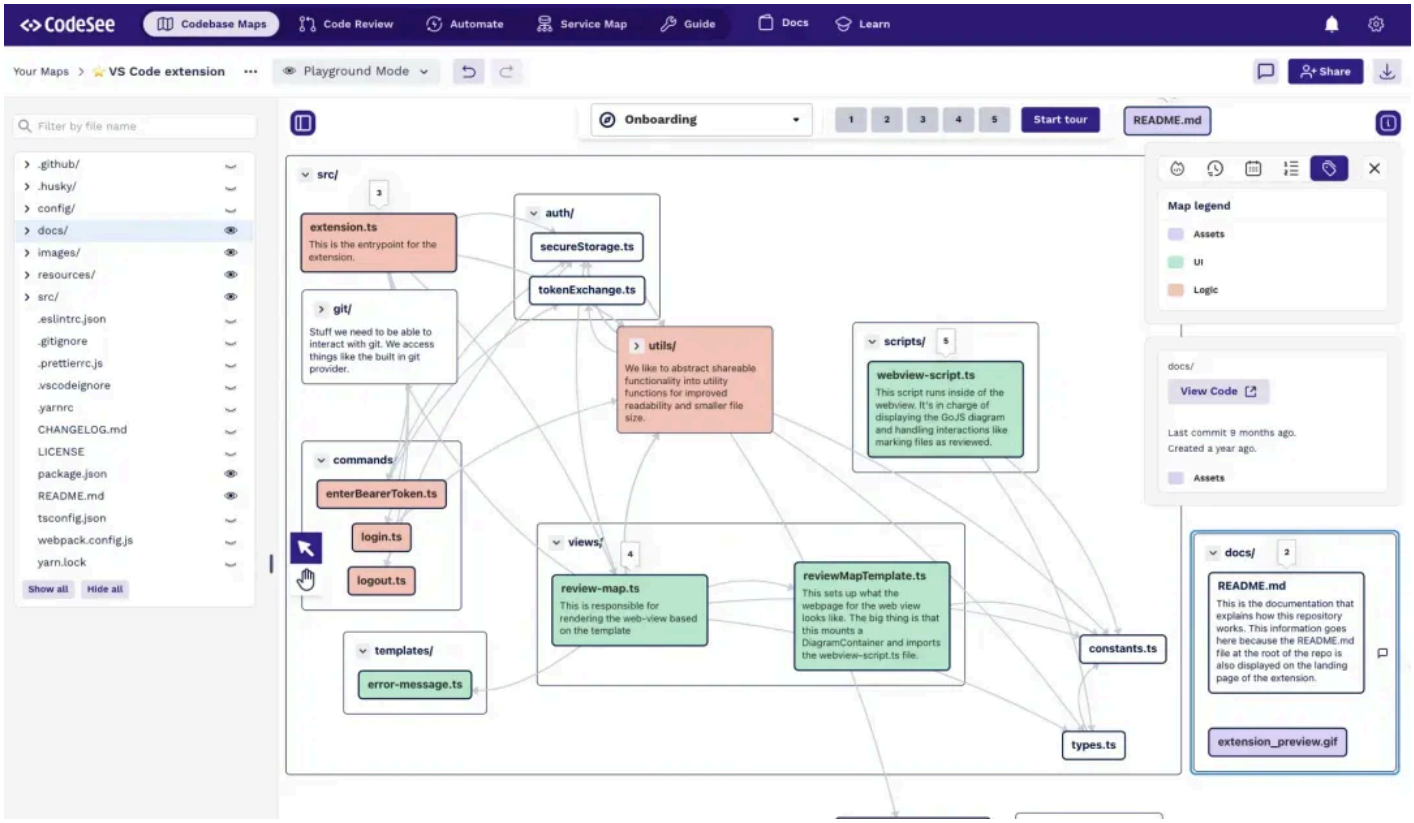
Let's look at how you can create these diagrams using CodeSee. CodeSee is a code visualization tool, but it also allows you to visualize at a higher level than pure code, giving you an understanding of how components and their containers interact.

Let's start with the containers. CodeSee Service Maps show you exactly that:

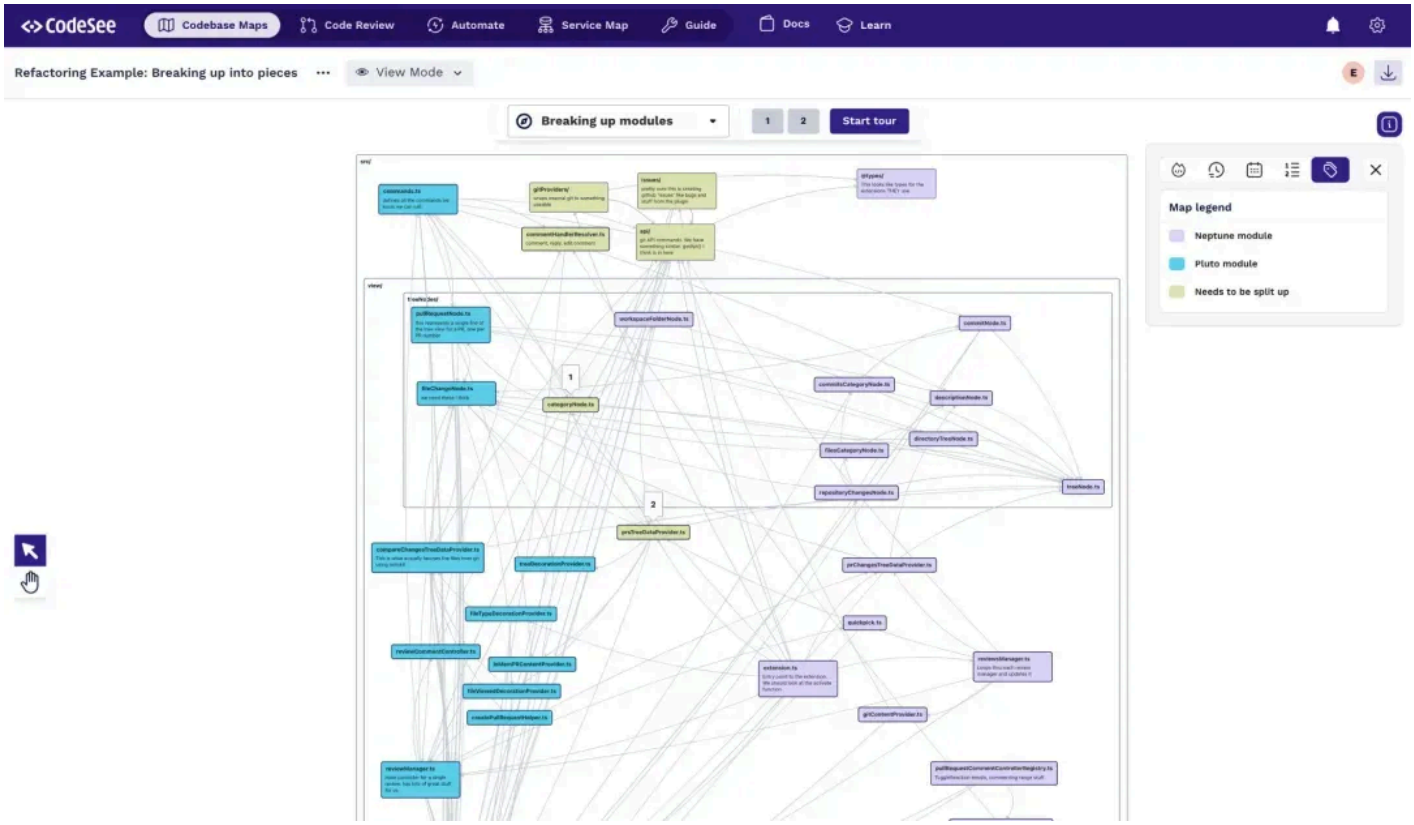


They show you every service, every container of code, within our system architecture. Here, for instance, we can see our frontend, databases, and services. You can think of this as a living representation of a container diagram. Whereas other visualization tools require you to create the visualizations of your code, CodeSee uses your codebase and services to create this for you.

The same is true at the component and code levels. Component diagrams in CodeSee can be illustrated through annotations of files and folders within your codebase:



At the code diagram level, you can see individual files and functions from within your codebase and how they interact:



Each of these offer the simplified view of your codebase and systems architecture that C4 diagrams are known for, but they are:

1. Automatically generated based on your code or service use.
2. Aren’t simplified too far, allowing your developers to get a real idea of how you code and systems work.
3. Are interactive, so developers or team leads can go into these visualizations and click down into the code to investigate further, or inspect different services to understand calls and errors.

CodeSee effectively gives you “living” C4 diagrams that change as you update services or push new code, and that you don’t have to spend hours creating yourself.

Limitations of C4 Diagrams

While the C4 model is a huge step forward from the traditional way of diagramming software architecture, it has some limitations:

- **Difficulty in maintaining consistency:** Achieving uniformity across multiple C4 diagrams within an organization is a challenge due to varying interpretations and practices. As systems evolve, diagrams must be continually updated, further complicating the task of maintaining consistency.
- **Interpretation differences:** C4 diagrams are prone to interpretation differences due to factors such as individual technical knowledge, familiarity with the system, and personal biases.
- **Not suitable for all types of systems:** The C4 diagram isn't applicable to all types of systems. It is less suitable for hardware-intensive systems, or systems that don't fit its hierarchical model.
- **Overkill for small systems:** For simpler systems, the layered abstraction may prove excessive, potentially complicating understanding.

Best Practices for Creating C4 Diagrams

Here are some strategies to ensure your C4 Diagrams are effective, clear, and accurate.

Keep it Simple

The primary purpose of a C4 Diagram is to facilitate understanding. Therefore, it's crucial to keep it simple and avoid overcomplication.

Strive to maintain clarity in your diagrams. Use straightforward language and avoid unnecessary jargon. Remember that your diagram may be used by various stakeholders, not all of whom may have the same technical knowledge.

Moreover, avoid overcrowding your diagram with excessive details. While it's essential to provide a comprehensive view, too much information can be overwhelming and obscure the main points.

Use Consistent Notation

Consistent notation is key to avoiding confusion and ensuring smooth communication. Decide on a set of symbols and conventions, and stick to them throughout all your diagrams.

This consistency allows for easier navigation and understanding, especially when dealing with multiple diagrams. It also minimizes interpretation differences and miscommunications, making for a more effective C4 Diagram.

Keep Diagrams Up-to-Date

Your C4 Diagrams should accurately reflect the current state of your system. As such, keeping them up-to-date is crucial. Using automated tools can make this much easier.

As your system evolves, so should your diagrams. Regularly revisit and revise your diagrams to ensure they mirror any changes or updates. This practice not only maintains the accuracy of your diagrams but also enables a better understanding and tracking of your system's evolution.

Include Legends

Legends are a valuable addition to any C4 diagram. They provide a quick reference for the symbols and notations used, aiding in the understanding of the diagram.

Including a legend in your diagram ensures all viewers are on the same page regarding the symbols' meanings. It reduces confusion and interpretation differences, making for a more effective diagram.

Use C4 Diagram Tools

Automated code visualization tools can streamline the process of creating C4 diagrams. These tools can scan your code and automatically create C4 diagrams at several levels.

Using such tools also ensures consistency in your diagrams, as the predefined notations and standards are maintained across all diagrams. Plus, many of these tools offer collaborative features, enabling better team coordination and communication.

Related Content: ***Learn more in our detailed guide to C4 model tools.***

System Architecture and Code Visualization with CodeSee

CodeSee helps developers better understand, navigate, and collaborate on their codebase. It captures and visualizes the flow and interactions of code within an application, allowing developers to see how different parts of the codebase connect and interact.

This makes it a great tool for understanding your system architecture and getting started with C4 diagrams. CodeSee starts the process for you by:

1. Generating an interactive map of your codebase and visualizing the flow of data and dependencies to help you understand the architecture and relationships between different components.
2. Enables you and your developers to add notes or annotations directly to the visualization, which can be useful for documentation, clarifying certain sections of the code, or communicating with team members.
3. Allowing you to navigate the visualization map to dive deep into specific areas, view relevant code snippets, and understand the context of different modules and components.
4. Integrating with version control so you can see how the codebase evolves over time, allowing developers to understand the impact of specific commits and see architectural changes across different versions.

Through these features, you can build up a continually updated, interactive, automated diagram of your code, containers, and components without ever having to draw out your entire system yourself. This real-time C4 option means less work for you and your team, while giving them more information about your system’s architecture.